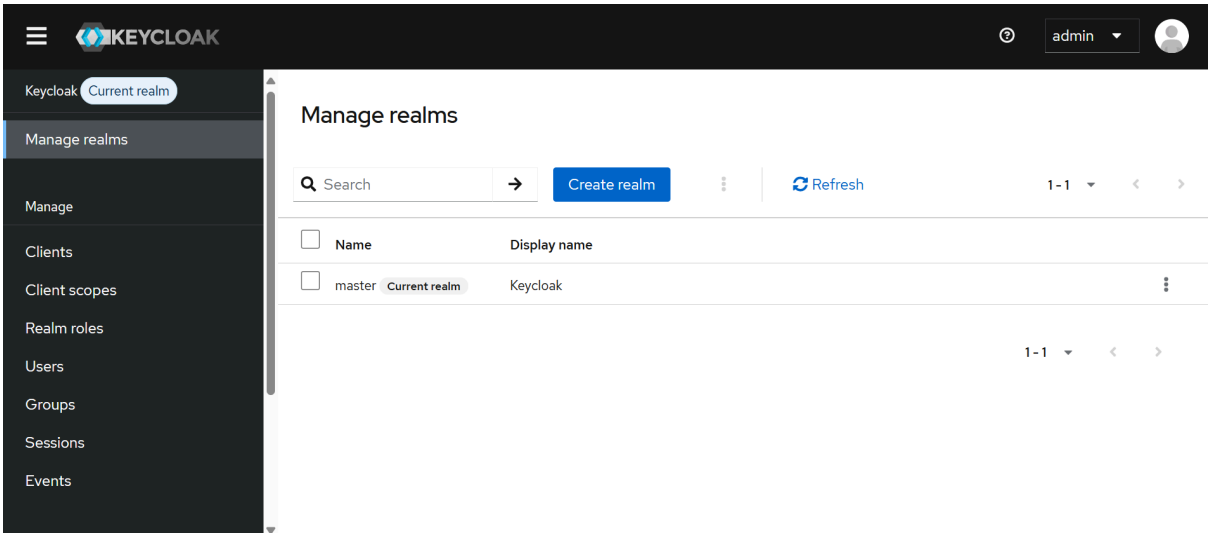
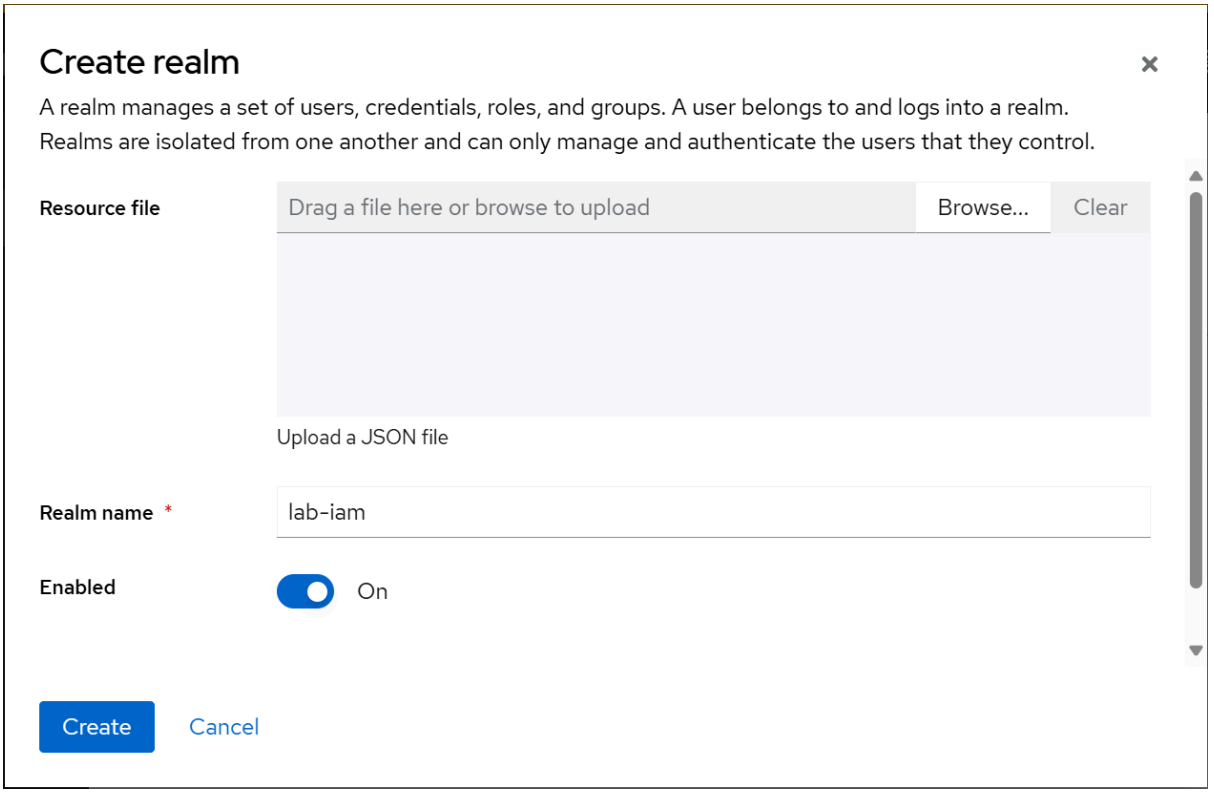
Étapes du TP

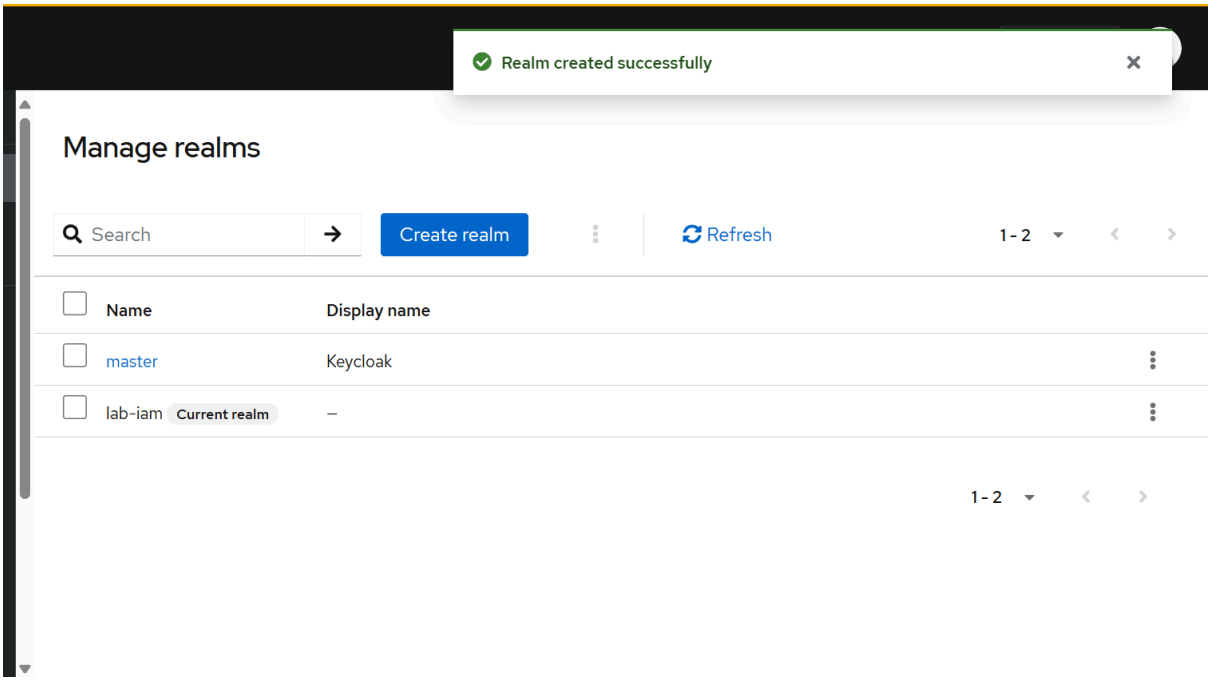
1. Déploiement rapide (Docker)



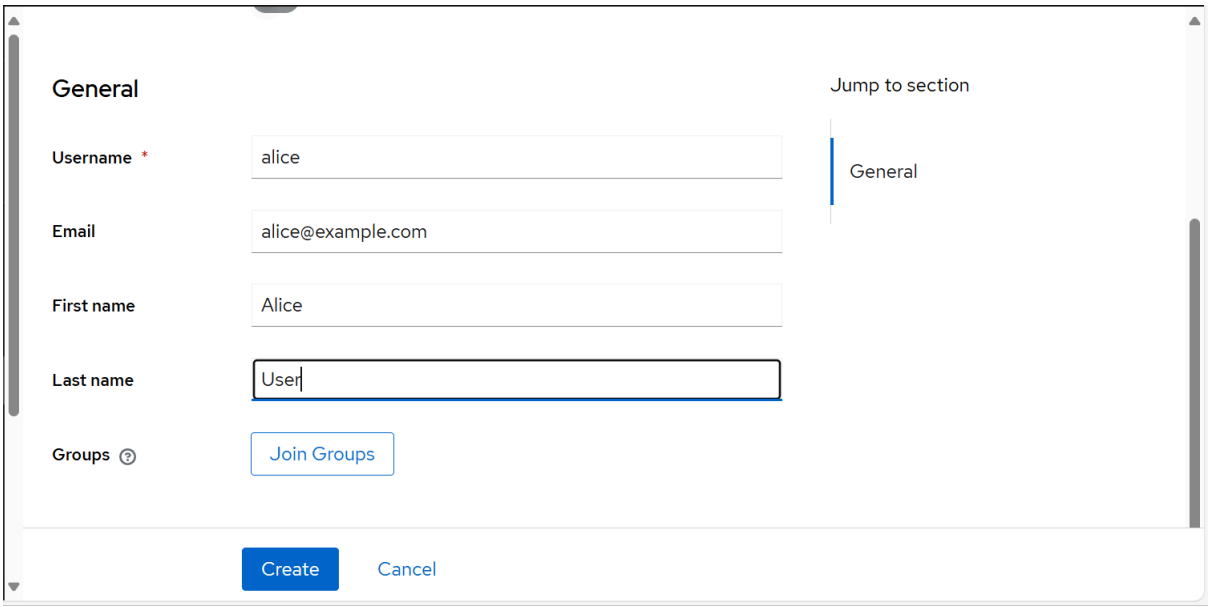
2. Créer le realm et un utilisateur
    1. Dans la console, cliquer sur Manage realms > Create Realm



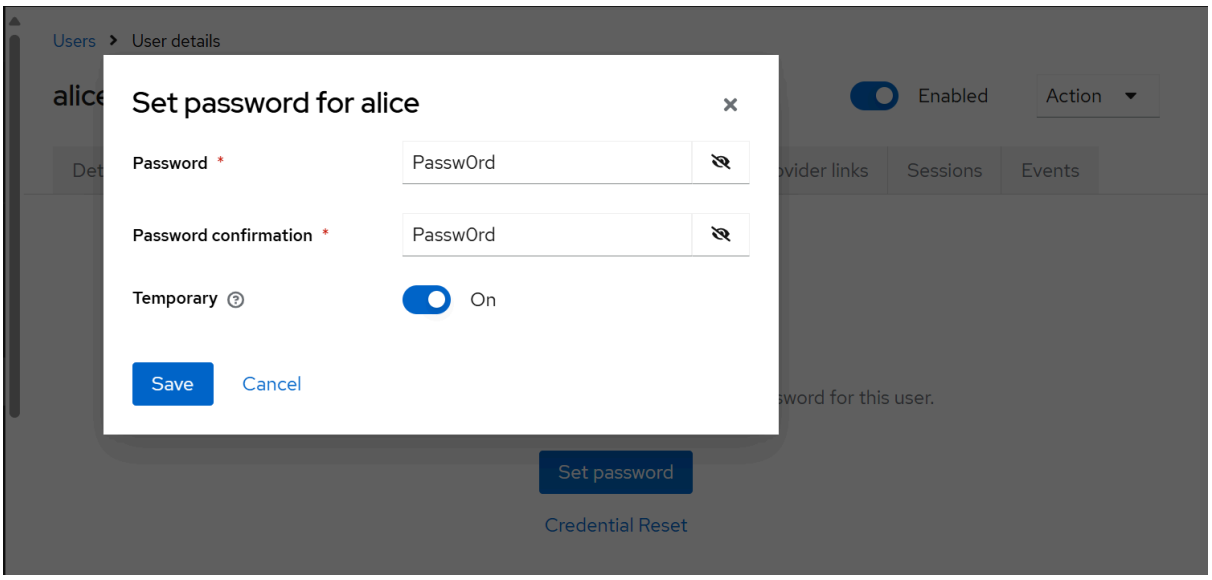
    2. Nommer le realm : lab-iam
    3. Valider



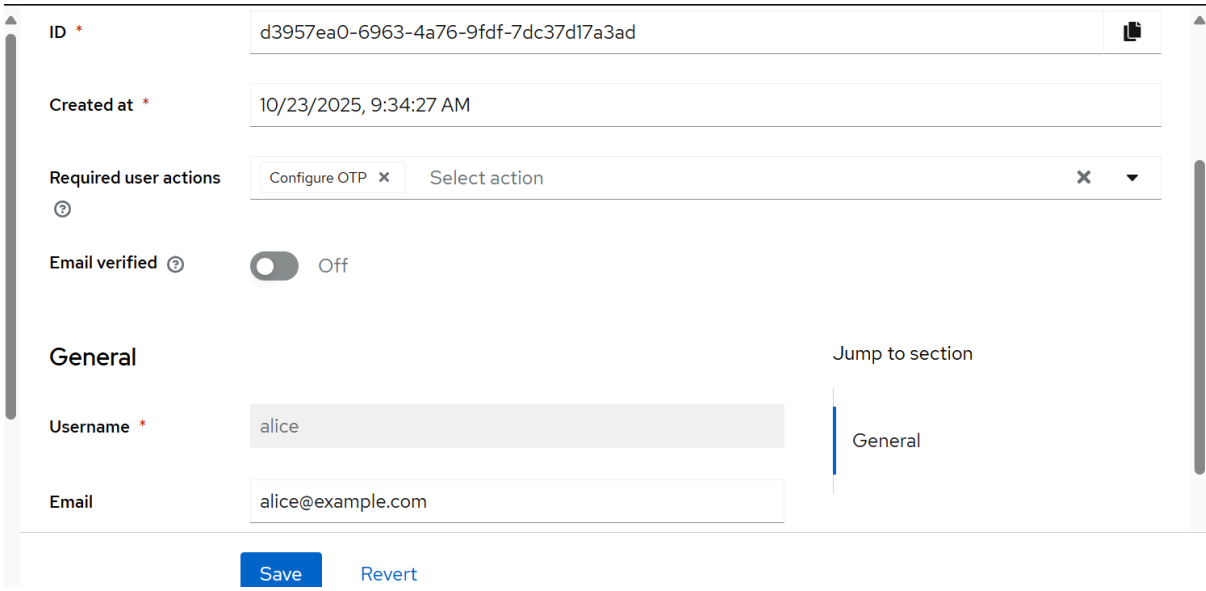
3. Création d'un utilisateur
    1. Aller dans Users → Add user.



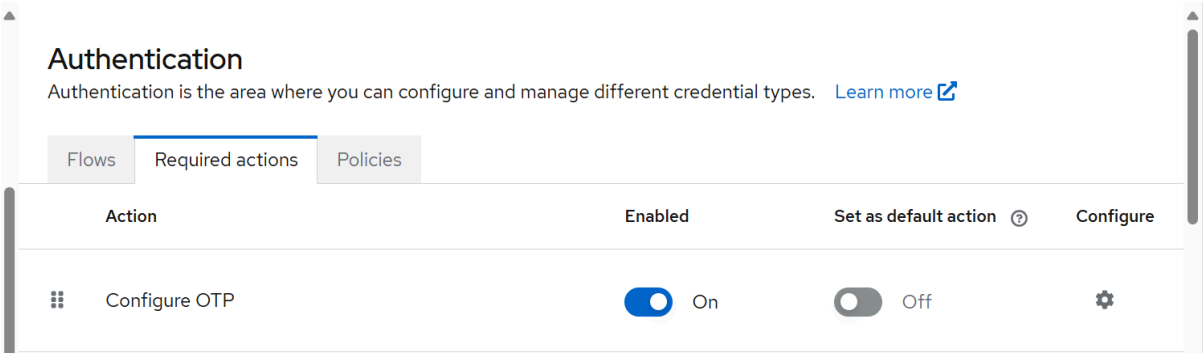
4. Aller dans l'onglet Credentials → définir un mot de passe (ex. Passw0rd!).



a. Cocher Temporary = OFF pour éviter de forcer le changement.



4. Configuration du MFA (OTP)



5. Sur le smartphone, ouvrir une application OTP (Google Authenticator, Microsoft Authenticator, Authy, etc.).

6. Ajouter un nouveau compte → scanner le QR code affiché.

.

aaaaaaaaaaa

Mobile Authenticator Setup

1. Install one of the following applications on your mobile:

   FreeOTP

   Microsoft Authenticator

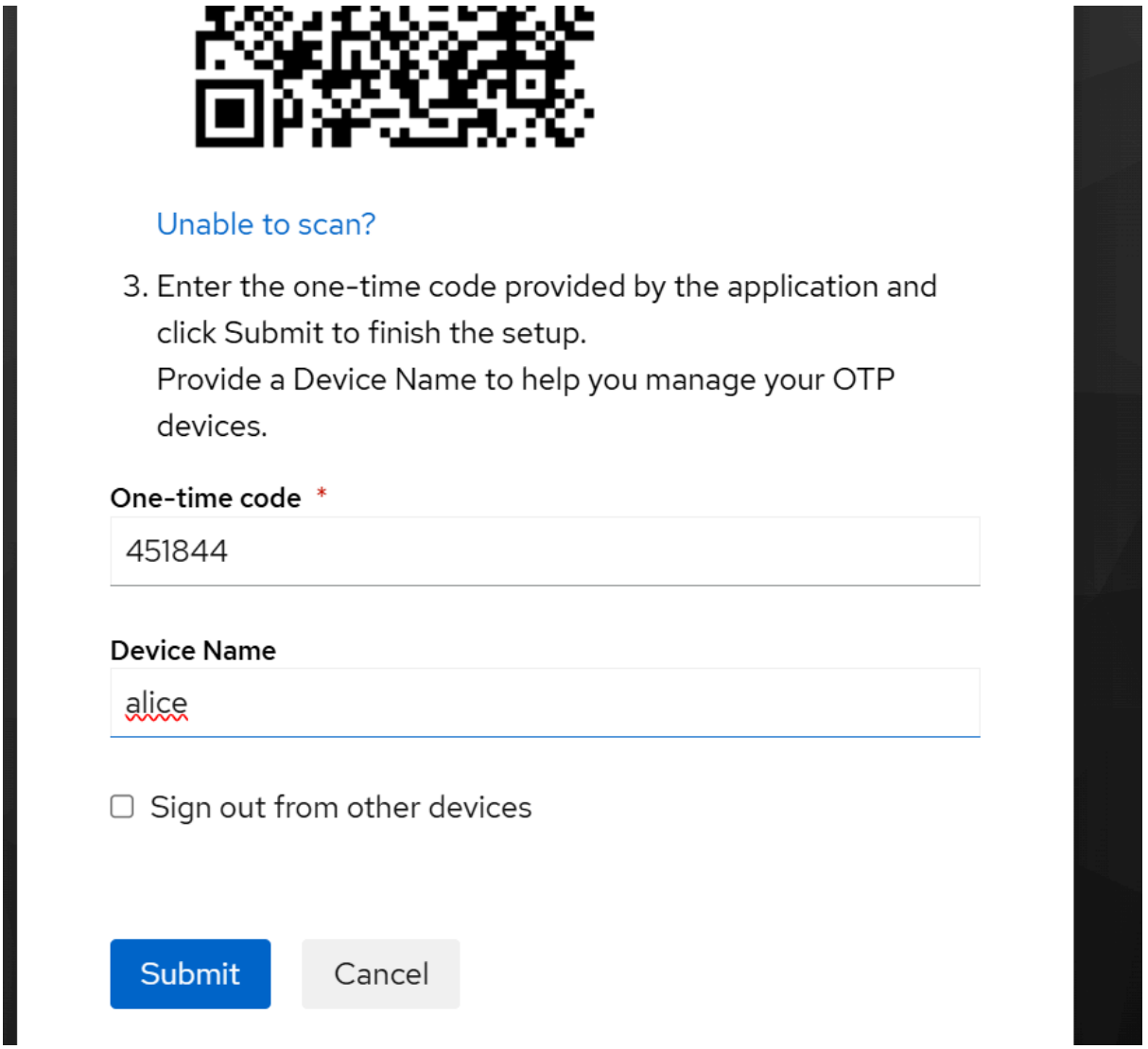
   Google Authenticator

2. Open the application and scan the barcode:

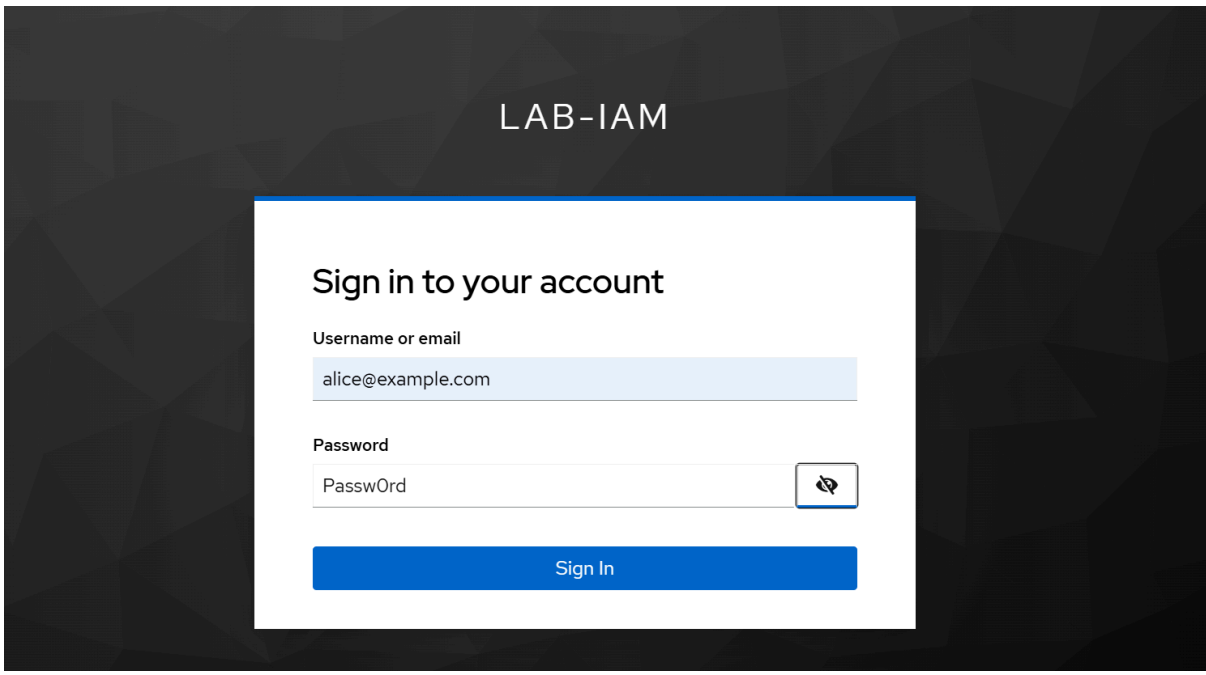
   Unable to scan?

3. Enter the one-time code provided by the application and click Submit to finish the setup.

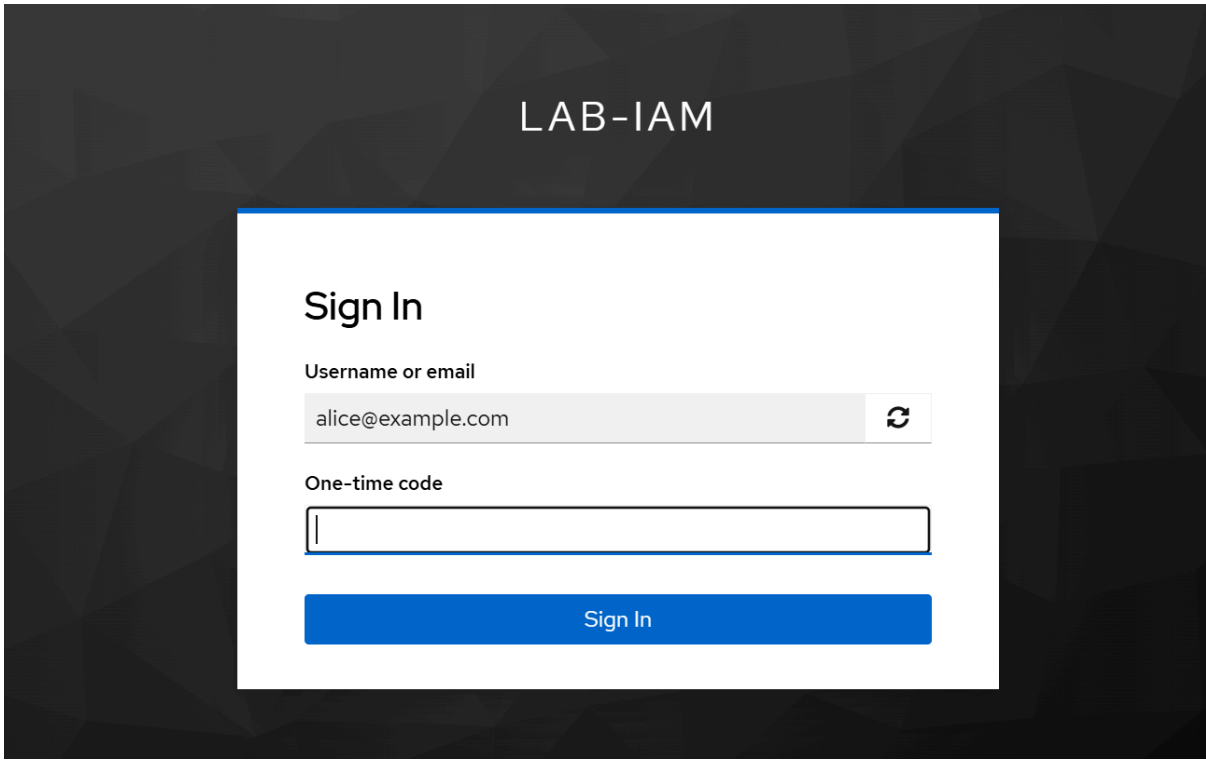
7. L’application génère un code à 6 chiffres, renouvelé toutes les 30 secondes



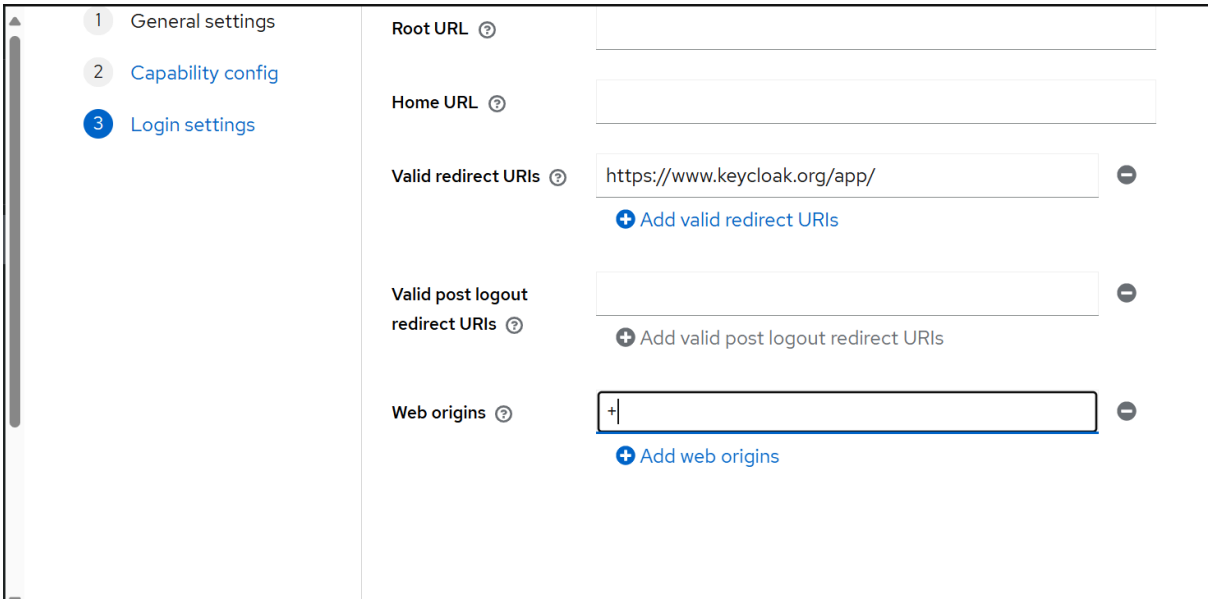
9. Cliquer sur Submit.
10. Keycloak enregistre la configuration MFA pour Alice.
11. Alice est maintenant connectée au portail utilisateur.
12. Déconnecter Alice puis se reconnecter.
13. Le processus d'authentification se déroule en deux étapes :
    a. Mot de passe



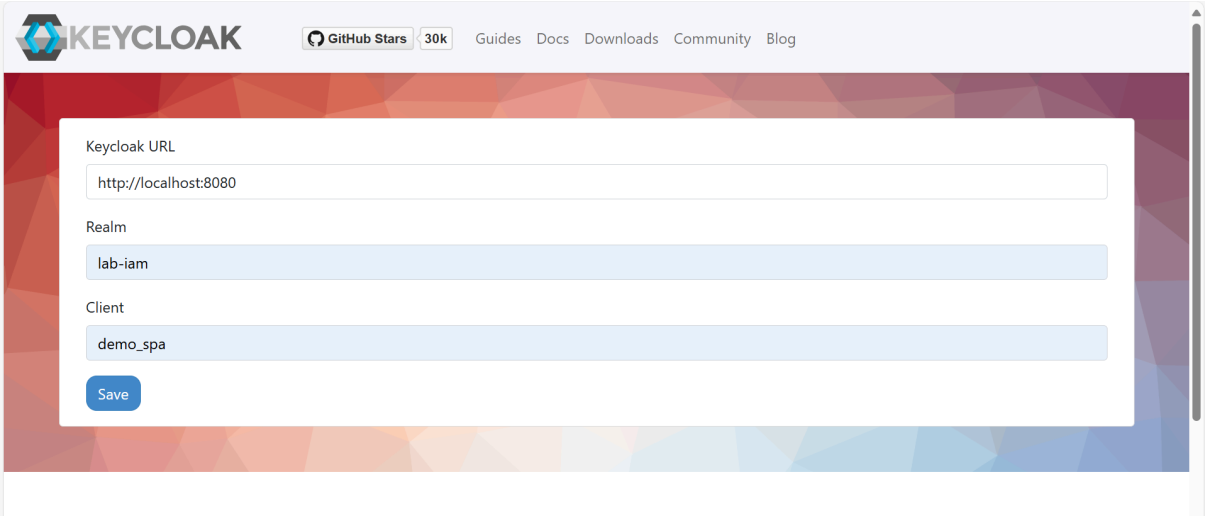
b. Code OTP (nouveau code généré par l'application) La connexion est accordée uniquement si les deux facteurs sont corrects.



5. Créer un client OIDC "demo-spa"

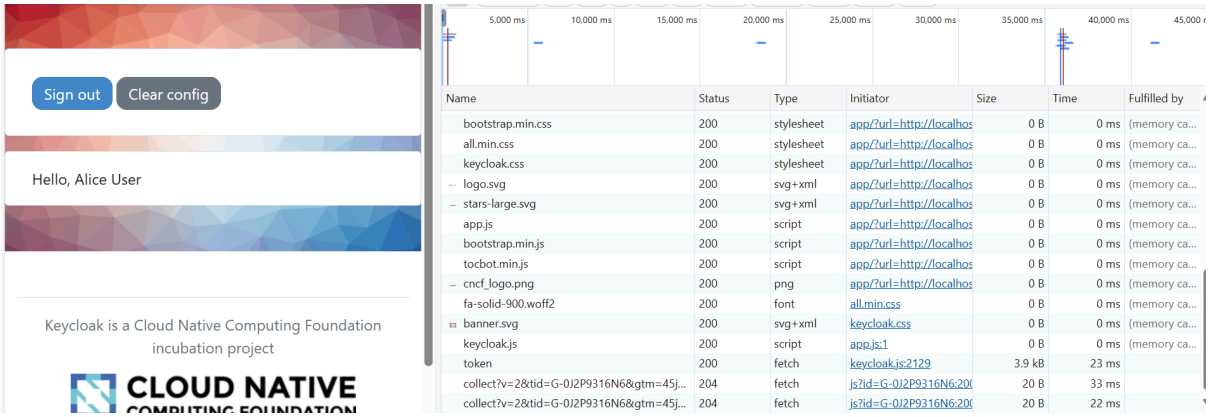


6. Tester le login OIDC
    1. Ouvrir https://www.keycloak.org/app/.
    2. Mettre à jour les champs Realm et Client.
    3. Cliquez sur Enregistrer pour utiliser la configuration du TP.



4. Cliquez sur Se connecter pour authentifier cette application en utilisant le serveur Keycloak que vous avez lancé plus tôt. 5. Vous êtes redirigé vers la page de login Keycloak.
5. Vous êtes redirigé vers la page de login Keycloak.
    - Saisir les identifiants de l'utilisateur alice.
    - Comme la MFA est activée, Keycloak demande aussi le code OTP généré par l'application Authenticator. Une fois validé → retour automatique sur l'application de test.

TOKENS



EN-TÊTE DÉCODÉ

JSON TABLEAU DES RÉCLAMATIONS COPIER


```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "SgCer0D5DMBe82S3dY4B9pmcAxUmiQ07HtEAH2-4yK8"
}
```

CHARGE UTILE DECODEE

JSON TABLEAU DES RÉCLAMATIONS COPIER


```
{
  "exp": 1761209290,
  "iat": 1761208990,
  "auth_time": 1761207653,
  "jti": "onrtac:6633ebc9-ecd5-8b17-e629-3f38d5deaa3d",
  "iss": "http://localhost:8080/realms/lab-iam",
  "aud": "account",
  "sub": "d3957ea0-6963-4a76-9fdf-7dc37d17a3ad",
  "typ": "Bearer",
  "azp": "demo_spa",
  "sid": "acb16086-6c95-677c-7276-2eb6717e2ef2",
  "acr": "0",
  "allowed-origins": [
    "https://www.keycloak.org"
  ],
```


VÉRIFICATION DE LA SIGNATURE JWT (FACULTATIF)

Entrez la clé publique utilisée pour signer le JWT ci-dessous :

CLÉ PUBLIQUE COPIER CLAIR

Clé publique valide


```
{
  "e": "AQAB",
  "kty": "RSA",
```


Format de la clé publique JWK

8. Questions :

1. Observation des tokens

a. Types de tokens récupérés :

- **access_token** : Sert à accéder aux API protégées. Il prouve que l'utilisateur est authentifié et autorisé à accéder à certaines ressources.

- **id_token** : Contient les informations d'identité de l'utilisateur (nom, prénom, email, etc.). Il est destiné au client pour identifier l'utilisateur connecté.

- **refresh_token** : Permet de renouveler un access_token expiré sans redemander la connexion de l'utilisateur.

Le token fourni correspond à un **access_token** (car il contient des informations de sécurité comme `realm_access`, `resource_access`, `scope`, etc.).

b. Durée de vie :

- `iat` : 1761208990 → émission

- `exp` : 1761209290 → expiration
  Durée de vie de l'access_token : 5 minutes (300 secondes)
  Durée de vie du refresh_token : généralement entre 30 minutes et plusieurs heures (selon la configuration Keycloak)

---

2. Analyse du contenu du id_token

a. sub (subject) :
`"sub": "d3957ea0-6963-4a76-9fdf-7dc37d17a3ad"`
→ Identifiant unique de l'utilisateur Alice dans Keycloak.

b. Attributs d'identité :

- `"preferred_username": "alice"`

- `"email": "alice@example.com"`

- `"name": "Alice User"`

- `"given_name": "Alice"`

- `"family_name": "User"`

c. Informations relatives aux rôles / permissions :
Oui, elles sont présentes :

```
"realm_access": {
  "roles": [
    "offline_access",
    "default-roles-lab-iam",
    "uma_authorization"
  ]
}
```

3. Analyse du contenu du access_token

a. Informations de sécurité incluses :

```
"scope": "openid profile email",
"realm_access": {
  "roles": ["offline_access", "default-roles-lab-iam",
"uma_authorization"]
},
"resource_access": {
  "account": {
    "roles": ["manage-account", "manage-account-links",
"view-profile"]
  }
}
```

Contient donc :

- **Scopes** : openid, profile, email
- **Rôles du realm** : offline_access, default-roles-lab-iam, uma_authorization
- **Rôles d'application (resource_access)** : manage-account, manage-account-links, view-profile

b. Utilisation par une API :
L'API reçoit le token dans l'en-tête HTTP :

```
Authorization: Bearer <access_token>
```

Cela lui permet de vérifier la validité du token et de contrôler les droits d'accès selon les rôles/scopes inclus.

4. Sécurité et gestion

a. Pourquoi l'application a besoin d'un refresh_token ?
Parce que l'access_token expire rapidement (5 minutes ici).
Le refresh_token permet de renouveler automatiquement un nouveau access_token sans que l'utilisateur ne doive se reconnecter.

b. Conséquences si un attaquant vole un refresh_token :
Très grave ❗
L'attaquant pourrait générer de nouveaux access_tokens valides et accéder aux ressources protégées **pendant toute la durée de vie du refresh_token**, compromettant totalement le compte.

---

5. Comparaison pratique

Élément	id_token	access_token
Contenu	Informations d'identité (nom, email, etc.)	Informations d'autorisation (rôles, scopes, permissions)
Utilisation	Pour identifier l'utilisateur côté client (ex : affichage "Bienvenue Alice")	Pour accéder à une API protégée
Exemple concret	L'application front-end affiche le profil de l'utilisateur	Une API vérifie si l'utilisateur a le droit de consulter `/api/orders`

Résumé global

Élément	Valeur
Type de token	Access Token
sub	d3957ea0-6963-4a76-9fdf-7dc37d17a3ad
Nom complet	Alice User
Username	alice
Email	alice@example.com
Rôles (realm)	offline_access, default-roles-lab-iam, uma_authorization
Rôles (application "account")	manage-account, manage-account-links, view-profile
Scopes	openid, profile, email

Durée de vie access_token	5 minutes
Durée de vie refresh_token	30 min à plusieurs heures (selon configuration)